Model Selection for Prediction

Stat 230 - Spring 2026

Overview

In class, we built a model to *understand* the relationship between ridership, weekends, and holidays. In this activity, you will build models to **predict**: a bike-share company wants to forecast the number of `rides` on days it hasn't seen yet. They do not care about the relative importance or significance of the predictor variables; they only care about how well the model can predict future days.

For prediction, a "good" model is one that does well on *new* data, not one that has the smallest p-values or the best fit to the data we already have.

We will compare six candidate models of increasing complexity. Run the code below to load the packages and the data, and set up the models.

```
library(dplyr)
library(purrr)
library(broom)
library(rsample)
library(yardstick)
library(parsnip)
library(workflows)
library(tune)

bikes <- read.csv("https://aluby.github.io/stat230/data/bikes.csv")
n <- nrow(bikes)

models <- list(
  M1 = rides ~ temp_actual,
  M2 = rides ~ temp_actual + humidity + windspeed,
  M3 = rides ~ temp_actual + humidity + windspeed + weekend + holiday + year,
  M4 = rides ~ temp_actual + humidity + windspeed + weekend + holiday + year +
season,
  M5 = rides ~ (temp_actual + humidity + windspeed + weekend + year +
season)^2,
  M6 = rides ~ (temp_actual + humidity + windspeed + weekend + year +
season)^3
)
```

Part 1: Judging models by the data used to fit them

```
m1 <- lm(rides ~ temp_actual, data = bikes)
m1_aug <- augment(m1)
rmse(m1_aug, truth = rides, estimate = .fitted)
```

```
# A tibble: 1 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard       1286.
```

```
bikes_fits <- map(models, \(f) lm(f, data = bikes))

full_rmse <- tibble(
  model = names(models),
  k = map_int(bikes_fits, \(m) length(coef(m))),
  rmse = map_dbl(bikes_fits, \(m) rmse(augment(m), truth = rides, estimate
= .fitted)$.estimate)
)
full_rmse
```

```
# A tibble: 6 × 3
  model     k  rmse
  <chr> <int> <dbl>
1 M1        2 1286.
2 M2        4 1185.
3 M3        7  792.
4 M4       10  662.
5 M5       34  609.
6 M6       74  550.
```

A. What happens to the RMSE as the models get more complex? Would you trust this table to tell you which model will predict *new* days best? Why or why not?

> The RMSE decreases every time, from about 1286 for M1 down to 550 for M6. This is guaranteed: adding terms can never make the fit to the *same* data worse. So this table would always crown the most complex model (M6), whether or not the extra terms capture real patterns or just noise in these particular 500 days. It measures how well the model fits data it has already seen, not how well it will predict new days, so we can't trust it for choosing a prediction model.

Part 2: Training and test data

```
set.seed(75)
```

```
bikes_split <- initial_split(bikes, prop = 0.8)
bikes_split
```

```
<Training/Testing/Total>
<400/100/500>
```

```
bikes_train <- training(bikes_split)
bikes_test  <- testing(bikes_split)
```

B. Fill in the blanks so that each model is **fit on the training data** and then used to predict the **test data**. The code computes the RMSE on both datasets for every model. Record the results.

```
train_test_rmse <- map_dfr(models, \(f) {
  m <- lm(f, data = bikes_train)                    # fit on the TRAINING
data
  tibble(
    train = rmse(augment(m), truth = rides, estimate = .fitted)$.estimate,
    test  = rmse(augment(m, newdata = bikes_test), truth = rides, estimate
= .fitted)$.estimate  # predict the TEST data
  )
}, .id = "model")

train_test_rmse
```

```
# A tibble: 6 × 3
  model train  test
  <chr> <dbl> <dbl>
1 M1    1264. 1370.
2 M2    1165. 1265.
3 M3     789.  807.
4 M4     662.  683.
5 M5     600.  659.
6 M6     525.  697.
```

C. Answer the following using your table.

a. As the models get more complex, what happens to the **training** RMSE? What happens to the **test** RMSE?
b. Which model has the smallest test RMSE?
c. When a model is **overfit**, it fits the data used to build it very well but predicts new data poorly. Which model(s) appear to be overfit? Which appear to be **underfit** (too simple to capture the patterns in the data)? Explain using the gap between the training and test RMSE.

a. The training RMSE decreases with every model (from 1264 to 525), just like in Part 1. The test RMSE decreases at first (1370, 1265, 807, 683, down to 659 for M5), but then **goes back up** for M6 (697).

b. M5 has the smallest test RMSE (658.7), though M4 (682.9) is close.

c. M6 is overfit: its training RMSE is the smallest (525) but its test RMSE is worse than M4's and M5's, and the gap between test and training RMSE is about 172, compared with only about 21 for M4. It has so many terms (74 coefficients) that it is fitting noise specific to the training days. M5's gap (59) is already starting to grow, so it may be slightly overfit as well. M1 and M2 are underfit: their errors are large on *both* datasets (about 1,200 to 1,400 rides), so they are missing important structure (they leave out `year`, `weekend`, and `season`, which carry a lot of the signal).

Part 3: These results depend on the random seed

Students choose their own seed; the answers below use `set.seed(2026)` as an example.

```
set.seed(2026)
bikes_split <- initial_split(bikes, prop = 0.8)
bikes_split
```

```
<Training/Testing/Total>
<400/100/500>
```

```
bikes_train2 <- training(bikes_split)
bikes_test2  <- testing(bikes_split)

train_test_rmse2 <- map_dfr(models, \(f) {
  m <- lm(f, data = bikes_train2)
  tibble(
    train = rmse(augment(m), truth = rides, estimate = .fitted)$.estimate,
    test  = rmse(augment(m, newdata = bikes_test2), truth = rides, estimate
= .fitted)$.estimate
  )
}, .id = "model")

train_test_rmse2
```

```
# A tibble: 6 × 3
  model train  test
  <chr> <dbl> <dbl>
1 M1    1280. 1310.
2 M2    1160. 1283.
3 M3     784.  824.
```

```
4 M4     663.  668.
5 M5     600.  685.
6 M6     532.  752.
```

D. Report the *seed you used* and your new *RMSE table*. Which model has the smallest RMSE on the training set? What about the test set?

> Seed 2026. M6 has the smallest training RMSE (532); this will be true for **any** seed, since the training RMSE always decreases as terms are added. The smallest test RMSE belongs to **M4** (667.7). The test winner depends on the seed: M4 and M5 are close enough that either can come out on top, and the other seeds I tried gave M4 (seeds 2 and 2026), M5 (seeds 1 and 75), and M6 (seed 230). Students' answers will vary. Full credit for a correct table, a seed, and M6 for training RMSE.

E. Is your answer to Q3(b) (from the first split) the same as your answer to Q4? What does this tell you about using a single train/test split to choose a model?

(The question numbers are out of date: "Q3(b)" is C(b) and "Q4" is D.)

> With seed 75, M5 had the smallest test RMSE; with seed 2026, M4 did. Students who happen to get the same winner twice should still note that this is partly luck. Different random splits can choose different "best" models when models are close (here M4 and M5 are within a few dozen rides of each other), so a single train/test split gives one noisy estimate of prediction error and is not a reliable way to choose between close models. Averaging over many splits (or cross-validation) gives a more stable answer.

Part 4: Compare to AIC and BIC

```
ic <- map_dfr(models, \(f) {
  glance(lm(f, data = bikes_train)) |>
    select(adj.r.squared, AIC, BIC)
}, .id = "model")

ic
```

```
# A tibble: 6 × 4
  model adj.r.squared   AIC   BIC
  <chr>         <dbl> <dbl> <dbl>
1 M1            0.339 6855. 6867.
2 M2            0.435 6794. 6814.
3 M3            0.739 6488. 6520.
4 M4            0.815 6353. 6397.
```

```
5 M5            0.838 6323. 6463.
6 M6            0.861 6295. 6595.
```

F. Which model does AIC prefer? Which does BIC prefer? How does this compare with the training/test set approach?

(Using the seed-75 training data.) **AIC prefers M6** (6295.2, the smallest), even though M6 was overfit according to the test set. **BIC prefers M4** (6396.9), which is close to the test-set winner (M5, with M4 close behind). So BIC's choice agrees much better with the train/test results, while AIC's choice is the overfit model. (Adjusted R^2, which has no real penalty for complexity, also keeps increasing and picks M6.)

The reason is the penalty. AIC charges 2 per coefficient, which is cheap relative to the large drop in SSE from adding 40 more terms, so M6 still looks good. BIC charges $\log(n)$ per coefficient (about 6 here), so M6's 74 coefficients are penalized much more heavily.

G. Suppose two different people use this data:

- A bike-share company wants to **forecast** next week's ridership to decide how many bikes to put out.
- A city planner wants to **understand** how weekends and holidays affect ridership, after accounting for the weather.

Which of the six models (or what kind of model) would you recommend to each person? Do they necessarily need the same model? Explain, referring to your results above.

Forecasting: I'd recommend M4 (or M5). They have the smallest, nearly tied test RMSE, and M4 is chosen by BIC. M4 is much simpler than M5, so it is less at risk of overfitting. The goal is accurate predictions on new days, so we don't need to interpret every coefficient, and a flexible model like M5 would also be defensible. We should avoid M6, which is overfit.

Understanding: I'd recommend a smaller, interpretable model like M3 or M4 (these include `weekend` and `holiday`, and adjust for weather, `year`, and `season`). A model with dozens of interaction terms has no single "weekend effect" or "holiday effect" to interpret, since each coefficient depends on the values of many other variables. For inference we choose the model based on the research question and interpretability, not just on prediction error.

So they don't necessarily need the same model: the best model depends on the goal. A model that is best for prediction may be too complex to interpret, and a model that is good for interpretation may leave some predictive accuracy on the table. (In this example the answer happens to be similar, M4, because the simpler model predicts nearly as well as the complex one.)

Function quick reference

Function	Purpose
`set.seed()`	Make random choices reproducible
`initial_split()`	Randomly split data into training and test sets
`training()`, `testing()`	Extract the training or test data from a split
`lm(formula, data)`	Fit a linear model
`augment(model, newdata)`	Add fitted values or predictions (`.fitted`) to a data frame
`rmse(data, truth, estimate)`	Root mean squared error
`map()`, `map_dfr()`	Repeat a computation for each model in a list